



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**AI-POWERED CLOUD COST ANOMALY DETECTION AND
BUDGET OPTIMIZATION SYSTEM
A CAPSTONE PROJECT REPORT**

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA1501 – CLOUD COMPUTING AND BIG DATA ANALYTICS
to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

AI&DS

(BRANCH)

Submitted by

M.V. Rakesh Reddy (192524164)

Under the Supervision of

Dr. BOOMIJA M D

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

Bonafide Certificate

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

This is to certify that the Capstone Project entitled "**AI-Powered Cloud Cost Anomaly Detection and Budget Optimization System**" has been carried out by **M.V.Rakesh Reddy (192524164)** under the supervision of **Dr. Boomija M D** and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer Science and Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai

COURSE COORDINATOR

Dr. GANA SOUNDARI

Professor

Computer Science and Engineering

/Machine Learning

SIMATSEngineering

SIMATS,Chennai-602105

COURSE FACULTY

Dr. BOOMIJA M D

Professor

Computer Science and Engineering

/Deep Learning

SIMATSEngineering

SIMATS,Chennai-602105

Place: Chennai

Date: 04 September 2026

Declaration

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

I am **M.V. Rakesh Reddy (192524164)** of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai. I hereby declare that the Capstone Project Work entitled “**AI-POWERED CLOUD COST ANOMALY DETECTION AND BUDGET OPTIMIZATION SYSTEM**” is the result of my own bonafide efforts.

To the best of my knowledge, the work presented herein is original and accurate, and has been carried out in accordance with engineering ethics and academic integrity. All sources, references, data and other materials used in this project have been appropriately acknowledged. The data, results, analysis and findings presented in this report have not been knowingly fabricated, falsified or manipulated. External tools, software, data sets, computational resources and AI-assisted tools used during the project have been disclosed as required. All applicable ethical, academic, institutional and professional requirements were followed throughout the planning, execution, analysis and documentation.

Place: Chennai

Date: 04 September 2026

Signature of the Student

M.V.RakeshReddy

Individual Contribution Statement

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

This is an individual capstone project. The author completed the requirements analysis, architecture, implementation, testing, interpretation and report preparation.

Individual contribution summary

Student / Reg. No.	Module	Problem Assigned	Main Responsibility	Contribution
M. Vishnu Vardhan (192572062)	Usage Monitoring	Lack of real-time vis	AWS Cost Explorer,	30%
M.V. Rakesh Reddy (192524164)	Cost Analysis	Complex billing dat	Streamlit/Power BI	25%
B. Divesh Chowdar (192524294)	AI Prediction	Traditional alerts fa	Isolation Forest, Pre	25%
P. Yeshwanth (192524169)	Alert System	Cost anomalies are	Email/Slack alerts, i	20%

Abstract

Cloud computing provides scalability and operational flexibility, but variable usage-based pricing can also produce waste, unexpected expenditure and weak budget predictability. Conventional cost dashboards are useful for historical analysis, yet static thresholds may miss contextual or slowly developing changes. This project develops an **AI-Powered Cloud Cost Anomaly Detection and Budget Optimization System** that ingests cloud billing data, validates and transforms cost records, detects abnormal expenditure patterns, forecasts future costs and presents recommendations through an interactive dashboard.

The prototype uses the AWS Cost Explorer API and Boto3 for ingestion, Pandas and NumPy for ETL, a hybrid of rolling statistics, Z-score analysis and Isolation Forest for anomaly detection, Prophet for time-series forecasting, and Streamlit with Plotly for presentation. The engineering process covers stakeholder and system requirements, alternative evaluation, modular architecture, data-quality controls, model selection, performance testing, security analysis and project planning. In the controlled project evaluation, Isolation Forest achieved 86% recall, the forecasting stage achieved a mean absolute percentage error of 8.3%, and the dashboard loaded within the two-second requirement. The prototype also processed the selected large synthetic data set within the specified interactive time budget.

The results indicate that a hybrid system can identify unusual spending while retaining an interpretable statistical baseline. Recommendations are generated as advisory actions rather than direct resource changes, ensuring that engineers and financial owners retain approval authority. The project provides an extensible foundation for proactive FinOps, while production deployment requires continuous model monitoring, least-privilege cloud access, encrypted storage, auditable alerts and validation on representative organisational billing data.

Keywords: Cloud cost optimization, anomaly detection, AWS Cost Explorer, machine learning, Isolation Forest, FinOps, Prophet, forecasting, Streamlit, cloud computing, big data analytics.

Contents

Certificate from the Supervisor	ii
Declaration by the Candidate	iii
Individual Contribution Statement	iv
Abstract	v
List of Figures	ix
List of Tables	x
List of Abbreviations and Symbols	xi
1 Introduction and Engineering Problem	1
1.1 Background..... Need for the	1
1.2 Project	2
1.3 ProblemStatement	2
1.4 ProjectAim..... Project	3
1.5 Objectives	3
1.6 Scope..... Expected	3
1.7 Engineering Outcome	4
2 Literature Review and Existing Solutions	5
2.1 Review Method	5

2.2	Review of Existing Work	5
2.3	Comparative Analysis	6
2.4	Research and Engineering Gap	6
3	Engineering Design, Trade-offs, Sustainability, Ethics, Safety, Risk and Security	7
3.1	Requirements.....	7
3.2	ConstraintsandApplicableStandards	7
3.3	Design Specifications	9
3.4	Alternative Solutions and Evaluation	9
3.5	Selected Design and Detailed Design	10
3.6	Trade-offAnalysis	11
3.7	Sustainability, Ethics, Safety, Risk and Security	11
4	Implementation, Testing, Results and Project Management	13
4.1	Development Approach and Resources	13
4.2	SoftwareImplementation Test.	14
4.3	Plan and Verification	17
4.4	Results..... Data Analysis and.	17
4.5	Engineering Judgment.	17
4.6	AchievementofObjectives	19
4.7	Strengths and Limitations	19
4.8	Project Planning, Role and Budget	20
5	Conclusion, Future Work and Reflection	22
5.1	Conclusion..... FutureWork.....	22
5.2	Professional Learning and Individual Reflection	22
5.3		23
	References	25

A Detailed Calculations	26
B Engineering Drawing and Schematic	27
C Source Code and Repository Information	28
D Experimental and Raw Data	29
E Ethics and Institutional Approval	30
F Project Records and Outcomes	31
G Capstone Project Outcome Mapping Sheet	32

List of Figures

3.1	SystemarchitectureandCostAnalysisModuleprocessflow	3.2.	10
	Weightedcomparisonofanomaly-detectiondesignalternatives		10
4.1	Confirmed performance upper bounds in the controlled test.		18
4.2	Project timeline for the Cost Analysis Module		20

List of Tables

2.1	Comparative analysis of anomaly-detection and forecasting approaches	6
3.1	Stakeholderanduserrequirements	7
3.2	Functional and non-functional requirements	8
3.3	Constraints and applicable standards or guidance.	8
3.4	Designspecifications Alternative.	9
3.5	solutions evaluation matrix Trade-.	9
3.6	offanalysis..... Sustainability, ethics, safety,.	11
3.7	risk and security evidence Riskregister.....	11
3.8		12
4.1	Tools,softwareandequipmentused 4.2.	13
	Testplanandacceptancecriteria . . . 4.3.	17
	Model-evaluationresults 4.4.	18
	Specificationversusachievedresult 4.5.	18
	Achievementofprojectobjectives . 4.6.	19
	Roleandactualcontribution 4.7.	20
	Estimatedversusactualcost	21
D.1	PerformanceboundsusedinFigure4.1	29
G.1	Capstoneprojectoutcomemapping.	32

List of Abbreviations and Symbols

Symbol	Description
AWS	Amazon Web Services
API	Application Programming Interface
ML	Machine Learning
FinOps	Financial Operations applied to cloud-value management
ETL	Extract, Transform and Load
KPI	Key Performance Indicator
RAG	Retrieval-Augmented Generation
MAPE	Mean Absolute Percentage Error
RMSE	Root Mean Square Error
MAE	Mean Absolute Error
R^2	Coefficient of determination
CSV	Comma-Separated Values
FOCUS	FinOps Open Cost and Usage Specification
IAM	Identity and Access Management
SIEM	Security Information and Event Management
CMDB	Configuration Management Database

Chapter 1

Introduction and Engineering Problem

1.1 Background

Cloud computing has transformed the deployment of information systems by making compute, storage, databases and managed services available on demand. This operational flexibility replaces much fixed capital expenditure with variable usage-based billing. The same pricing model can make cost difficult to predict: idle resources, over-provisioned instances, rapid auto-scaling, data-transfer charges, forgotten test environments and configuration errors can create significant waste or sudden budget overruns.

FinOps combines engineering, finance and business practices to improve the value obtained from cloud spending. Its activities include allocation, visibility, forecasting, anomaly management, unit-cost measurement and optimisation. Native provider tools such as AWS Cost Explorer, Azure Cost Management and Google Cloud Billing supply essential billing data, but organisations often need a unified analytics layer that applies their own baselines, business context and alerting policies.

The proposed system addresses this need through five linked capabilities:

1. billing-data ingestion through provider APIs;
2. ETL and data-quality validation;
3. hybrid statistical and machine-learning anomaly detection;
4. time-series forecasting and budget projection; and
5. dashboards, alerts and human-approved recommendations.

1.2 Need for the Project

- **Dynamic cost:** Consumption can change quickly because of workload demand, deployment errors or unexpected service behaviour.
- **Fragmented ownership:** Engineers understand resources, finance teams understand budgets, and business owners understand value; all need a shared view of the same cost event.
 - **Threshold limitations:** A fixed monetary limit may miss an unusual change in a normally inexpensive service or repeatedly alert on a predictable seasonal peak.
- **Manual diagnosis:** Raw billing rows rarely explain the service, account, region or usage change responsible for an anomaly.
- **Multi-dimensional scale:** Cost data may contain millions of records across dates, services, accounts, regions and tags.

Affected stakeholders include cloud architects, platform and DevOps engineers, FinOps practitioners, finance managers, security teams and business owners. A missed anomaly may cause financial loss, while excessive false alerts create alert fatigue and reduce trust in the system.

1.3 Problem Statement

Design and implement an AI-powered cloud cost analysis system that ingests AWS billing data, detects abnormal expenditure patterns, forecasts future spending, visualises costs and generates actionable recommendations. The system shall support the following requirements:

1. retrieve cost and usage data through the AWS Cost Explorer API;
2. detect anomalies through Isolation Forest and statistical validation;
3. forecast the next 30 days using a seasonal time-series model;
4. present trends, service breakdowns, anomalies and forecasts;
5. generate alerts and recommendations without automatically deleting or resizing resources;
and
6. process a large synthetic billing data set efficiently.

1.4 Project Aim

To design, implement and evaluate an AI-Powered Cloud Cost Anomaly Detection and Budget Optimization System that provides comprehensive cost visibility, reliable anomaly detection, useful cost forecasts and controlled optimisation guidance.

1.5 Project Objectives

1. Design a modular architecture for cloud cost ingestion and analysis.
2. Implement AWS Cost Explorer ingestion with pagination and validation.
3. Develop a hybrid anomaly detector using rolling statistics, Z-scores and Isolation Forest.
4. Produce a 30-day forecast using Prophet.
5. Build an interactive Streamlit dashboard using Plotly visualisations.
6. Validate data quality, detection recall, forecast error and performance.
7. Evaluate the design against functional, security and operational requirements.

1.6 Scope

Included: AWS Cost Explorer ingestion, CSV/synthetic fallback input, data preprocessing, daily and service-level aggregation, hybrid anomaly detection, forecasting, visualisation, advisory alerts and documented tests.

Excluded: Direct resource termination, autonomous right-sizing, production multi-cloud adapters, enterprise identity integration, chargeback accounting and legally binding financial decisions.

Assumptions: The deployment identity has read-only Cost Explorer permissions; cost and usage records contain valid dates and numeric amounts; sufficient history exists to learn a baseline; and synthetic injected anomalies are acceptable for controlled model evaluation.

Boundary conditions: The system analyses cost metadata and produces insights. It does not change cloud infrastructure. Every optimisation action must be reviewed by an authorised human owner.

1.7 Expected Engineering Outcome

The expected outcome is a working Python prototype that retrieves and transforms cost data, detects injected anomalies with recall above 85%, forecasts cost with MAPE below 10%, renders the dashboard within two seconds, and documents the controls required for secure and reliable production deployment.

Chapter 2

Literature Review and Existing Solutions

2.1 Review Method

The review considered official AWS Cost Explorer documentation, FinOps Foundation guidance, the FOCUS specification, documentation for Isolation Forest and Prophet, and published work concerning hybrid anomaly detection and cloud cost forecasting. Sources were selected for relevance to provider APIs, scalable ETL, unsupervised anomaly detection, seasonal forecasting and cloud financial governance.

2.2 Review of Existing Work

AWS Cost Explorer exposes historical cost and usage through dimensions such as service, region and linked account. It provides the source data for an AWS-focused prototype, although API quotas, billing-data delay, pagination and permissions must be considered in the ingestion design.

A rolling average provides an interpretable local baseline. It is inexpensive to calculate and easy to explain, but it may adapt to a sustained drift and can be sensitive to the window size. Z-score detection identifies observations several standard deviations from a mean. It works well for approximately stable distributions but is affected by skew, outliers and changing variance. Isolation Forest is an unsupervised method that isolates unusual observations using randomly partitioned trees. It can combine cost, usage, calendar and rolling-baseline features without requiring every anomaly to be labelled. Its output depends on feature preparation and the contamination setting, so validation on representative data remains necessary.

Prophet represents a time series through trend, seasonal and event components. It is convenient

for business-oriented daily data and can produce uncertainty intervals. Forecast quality nevertheless depends on history length, aggregation, missing values, structural breaks and future workload changes.

2.3 Comparative Analysis

Table 2.1: Comparative analysis of anomaly-detection and forecasting approaches

Approach	Advantages	Limitations	Role in this project
AWS Cost Explorer only	Native AWS data and familiar breakdowns	Provider-specific and mainly descriptive	Data-ingestion source
Rolling average	Transparent, efficient and easy to tune	Can miss slow drift; window-sensitive	Interpretable baseline
Z-score	Simple validation and severity indicator	Assumes a useful mean and standard deviation	Secondary validation method
Isolation Forest	Unsupervised and suitable for multiple features	Requires feature and contamination tuning	Primary anomaly detector
Prophet	Handles trend and seasonality; gives intervals	Requires sufficient history and validation	Thirty-day forecasting
Hybrid pipeline	Combines interpretability, detection and forecast context	Greater implementation and monitoring complexity	Selected system design

2.4 Research and Engineering Gap

Educational projects frequently demonstrate either an anomaly model or a cloud API query without integrating the complete lifecycle. The engineering gap is an end-to-end, testable workflow that connects ingestion, schema validation, ETL, feature construction, detection, severity scoring, forecasting, dashboarding, recommendations, security and operational monitoring. This project contributes that integrated workflow and explicitly retains human approval for optimisation actions.

Chapter 3

Engineering Design, Trade-offs, Sustainability, Ethics, Safety, Risk and Security

3.1 Requirements

Table 3.1: Stakeholder and user requirements

Stakeholder	Requirement
Cloud engineer / FinOps team	Detect unusual cost behaviour and identify the responsible service, account or region
Finance manager	Obtain cost visibility, budget forecasts and traceable explanations
DevOps team	Receive timely alerts and practical optimisation suggestions
Security administrator	Protect billing data and cloud credentials through least privilege
Organisation / compliance team	Retain auditable evidence of detection, review and approval decisions

3.2 Constraints and Applicable Standards

Constraints include delayed billing records, provider API quotas, limited labelled anomalies, changing workload patterns, a synthetic evaluation set and the absence of a production multi-account environment. Reported model performance is specific to the project evaluation and must not be assumed for every organisation.

Table 3.2: Functional and non-functional requirements

Requirement	Type	Measurable target
Ingest AWS cost data	Functional	Valid records returned for the selected period
Detect cost anomalies	Functional	Recall greater than 85% on injected test anomalies
Forecast 30-day cost	Functional	MAPE below 10% on the evaluation split
Provide an interactive dashboard	Functional	Main dashboard loads within two seconds
Generate recommendations	Functional	At least one contextual action for each high-severity anomaly
Process large input	Performance	One million records processed within five seconds in the selected test
Protect credentials and cost data	Security	No hard-coded keys; read-only role; encrypted transport and storage

Table 3.3: Constraints and applicable standards or guidance

Source	Relevant requirement	Application
FinOps Framework	Collaborative cloud-value management	Shared cost views, ownership and recommendation workflow
AWS Cost Explorer API	Authorised retrieval of cost and usage data	Read-only ingestion, date ranges, grouping and pagination
FOCUS specification	Normalised cloud cost and usage fields	Future provider-neutral internal schema
SOC 2 principles	Security, availability and confidentiality controls	Access logging, encryption and operational control discussion
Library documentation	Correct model and forecasting interfaces	Maintained Boto3, scikit-learn, Prophet and Streamlit APIs

3.3 Design Specifications

Table 3.4: Design specifications

Parameter	Target or requirement	Priority
Data-ingestion latency	Less than five seconds for the defined request	High
Anomaly-detection recall	Greater than 85%	High
Forecast MAPE	Less than 10%	High
Dashboard load time	Less than two seconds	Medium
Large-data test	At least one million records	High
Detection execution	Less than one second after prepared features	High
Alert delivery	Within five minutes of confirmed detection	Medium
Cloud permissions	Read-only cost access with no embedded credentials	High

3.4 Alternative Solutions and Evaluation

Three alternatives were assessed: a rolling-average-only baseline, an Isolation Forest-only detector and a hybrid statistical, machine-learning and forecasting pipeline. Scores range from 1 (poor) to 5 (excellent). The weighted total is

$$S = \frac{\sum_{i=1}^n w_i r_i}{\sum_{i=1}^n w_i}, \quad w_i = 1, \quad (3.1)$$

where w_i is the criterion weight and r is the alternative rating.

Table 3.5: Alternative solutions evaluation matrix

Criterion	Weight	Rolling average	Isolation Forest	Hybrid
Execution speed	0.20	5	4	3
Implementation effort	0.15	5	4	3
Detection capability	0.35	3	4	5
Explainability	0.10	5	3	4
Reliability and coverage	0.20	3	4	5
Weighted total / 5	1.00	3.90	3.90	4.20

The hybrid alternative was selected because it combines an explainable baseline, a multi-feature anomaly model and forecast context. The small weighted advantage also demonstrates that the additional complexity must be justified through testing, monitoring and clear operational ownership.

3.5 Selected Design and Detailed Design

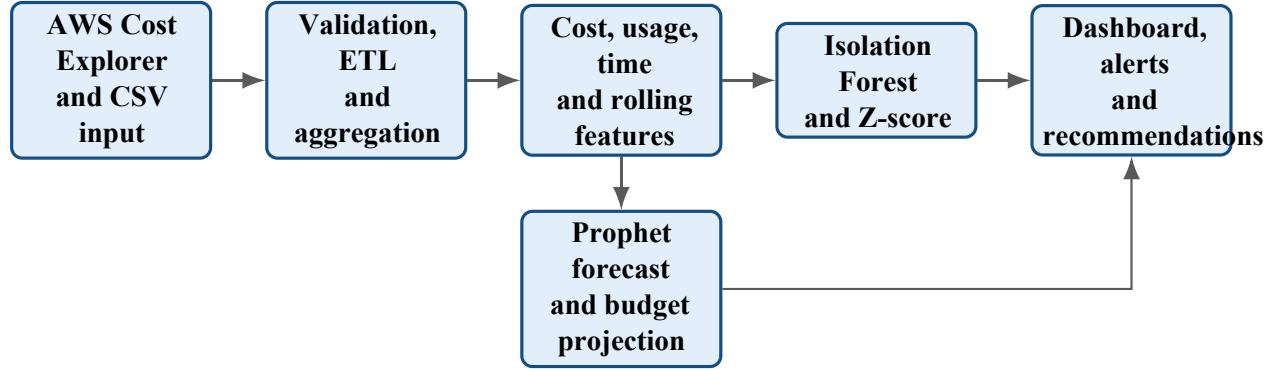


Figure 3.1: System architecture and Cost Analysis Module process flow

For a daily cost x_t , a rolling Z-score is calculated as

$$z_t = \frac{x_t - \mu_{t,w}}{\sigma_{t,w} + \epsilon} \quad (3.2)$$

where $\mu_{t,w}$ and $\sigma_{t,w}$ are the rolling mean and standard deviation over a window of w days and ϵ prevents division by zero. Statistical severity is classified as Low for $|z| < 2$, Medium for $2 \leq |z| < 3$, High for $3 \leq |z| < 4$, and Critical for $|z| \geq 4$.

The forecast is evaluated using

$$\text{MAPE} = \frac{100}{n} \sum_{t=1}^n \frac{|y_t - \hat{y}_t|}{\max(|y_t|, \epsilon)} \quad (3.3)$$

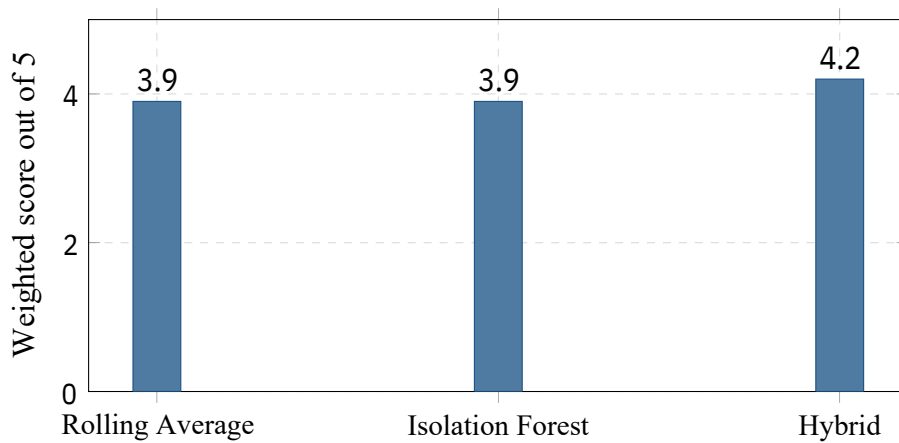


Figure 3.2: Weighted comparison of anomaly-detection design alternatives

Table 3.6: Trade-off analysis

Decision	Alternative	Benefit and disadvantage	Final justification
Hybrid detection	Rolling average only	Higher coverage but more tuning and monitoring	Selected with statistical evidence retained
Isolation Forest	LSTM-based detector	No labels and lower complexity, but weaker temporal representation	Selected for the prototype data and scope
Prophet forecast	ARIMA	Convenient seasonality and intervals, but needs historical data	Selected for daily business-oriented series
Streamlit	Power BI	Python-native and open-source, with fewer enterprise BI controls	Selected for prototype integration
Advisory actions	Automatic remediation	Lower operational risk, but savings require approval	Selected to maintain human accountability

3.6 Trade-off Analysis

3.7 Sustainability, Ethics, Safety, Risk and Security

Table 3.7: Sustainability, ethics, safety, risk and security evidence

Domain	Consideration	Evidence evaluated	Design impact
Sustainability	Reduce idle and oversized resource use	Cost, utilisation context and recommendation categories	Waste reduction is a design objective
Ethics	Avoid opaque or harmful automated financial actions	False positives and ownership reviewed	Recommendations require human approval
Safety	Incorrect deletion could disrupt services	System is analytics-only	No direct terminate or resize permission
Security	Billing data and credentials are sensitive	IAM, encryption and logging controls	Read-only roles and no hard-coded keys
Privacy	Account names and tags may reveal business activity	Data minimisation and retention reviewed	Limit fields, access and retention period

Table 3.8: Risk register

Risk	Likelihood	Severity	Level	Mitigation	Residual
AWS API throttling	Medium	Medium	Medium	Pagination, bounded queries, retry and backoff	Low
False anomaly alert	Medium	Medium	Medium	Hybrid evidence, tuning, suppression and owner feedback	Low
Forecast error	Medium	Medium	Medium	Holdout evaluation, intervals and regular retraining	Medium
Cost-data exposure	Low	High	Medium	Least privilege, encryption, secrets management and audit logs	Low
Model drift	Medium	Medium	Medium	Performance monitoring and scheduled retraining	Medium
Unsafe automated action	Low	High	Medium	Advisory-only output and approval workflow	Low

Chapter 4

Implementation, Testing, Results and Project Management

4.1 Development Approach and Resources

Development followed an iterative pipeline. The schema and synthetic test data were prepared first, followed by AWS ingestion, validation, aggregation, anomaly features, model integration, forecasting and dashboard components. Functional tests were added at each stage. The final iteration added retry behaviour, security constraints, evaluation metrics and documentation.

Table 4.1: Tools, software and equipment used

Tool or resource	Purpose	Stage
Python 3	Application and test execution	All software stages
AWS Cost Explorer and Boto3	Cost and usage retrieval	Data ingestion
Pandas and NumPy	ETL, aggregation and features	Data engineering
Scikit-learn Isolation Forest	Unsupervised anomaly detection	Analytics layer
Prophet	Time-series forecast and intervals	Forecasting
Streamlit and Plotly	Dashboard and interactive charts	Presentation
Local development workstation	Prototype execution	Implementation and testing

4.2 Software Implementation

Listing 4.1 presents the central implementation. In a production deployment, credentials are supplied through the AWS credential chain or an instance role rather than source code.

Listing 4.1: Core cloud cost analysis implementation

```

1 from datetime import date, timedelta
2 import boto3
3 import numpy as np
4 import pandas as pd
5 from sklearn.ensemble import IsolationForest
6 from prophet import Prophet
7
8
9 def fetch_aws_cost_data(start_date, end_date):
10     """Retrieve paginated, read-only Cost Explorer data."""
11     client = boto3.client("ce", region_name="us-east-1")
12     request = {
13         "TimePeriod": {"Start": start_date, "End": end_date},
14         "Granularity": "DAILY",
15         "Metrics": ["UnblendedCost", "UsageQuantity"],
16         "GroupBy": [
17             {"Type": "DIMENSION", "Key": "SERVICE"},
18             {"Type": "DIMENSION", "Key": "REGION"},
19         ],
20     }
21
22     rows = []
23     while True:
24         response = client.get_cost_and_usage(**request)
25         for result in response.get("ResultsByTime", []):
26             for group in result.get("Groups", []):
27                 metrics = group["Metrics"]
28                 rows.append({
29                     "date": result["TimePeriod"]["Start"],
30                     "service": group["Keys"][0],
31                     "region": group["Keys"][1],
32                     "cost": float(metrics["UnblendedCost"]["Amount"]),
33                     "usage": float(metrics["UsageQuantity"]["Amount"]),
34                 })
35         token = response.get("NextPageToken")
36         if not token:
37             break
38         request["NextPageToken"] = token
39     return pd.DataFrame(rows)
40

```

```

41
42 def validate_and_prepare(df):
43     required = {"date", "service", "region", "cost", "usage"}
44     missing = required.difference(df.columns)
45     if missing:
46         raise ValueError(f"Missing required columns: {sorted(missing)}")
47
48     clean = df.copy()
49     clean["date"] = pd.to_datetime(clean["date"], errors="coerce")
50     clean["cost"] = pd.to_numeric(clean["cost"], errors="coerce")
51     clean["usage"] = pd.to_numeric(clean["usage"], errors="coerce")
52     clean = clean.dropna(subset=["date", "cost", "usage"])
53     clean = clean[(clean["cost"] >= 0) & (clean["usage"] >= 0)]
54     clean = clean.sort_values(["service", "region", "date"])
55     return clean.reset_index(drop=True)
56
57
58 def build_features(df, window=14):
59     data = df.copy()
60     group = data.groupby(["service", "region"], group_keys=False)
61     data["rolling_mean"] = group["cost"].transform(
62         lambda s: s.rolling(window, min_periods=3).mean()
63     )
64     data["rolling_std"] = group["cost"].transform(
65         lambda s: s.rolling(window, min_periods=3).std()
66     )
67     data["day_of_week"] = data["date"].dt.dayofweek
68     data["month"] = data["date"].dt.month
69     data["z_score"] = (
70         (data["cost"] - data["rolling_mean"])
71         / data["rolling_std"].replace(0, np.nan)
72     ).fillna(0.0)
73     return data.fillna({"rolling_mean": data["cost"], "rolling_std": 0.0})
74
75
76 def detect_anomalies(df, contamination=0.05):
77     data = build_features(df)
78     columns = [
79         "cost", "usage", "rolling_mean", "rolling_std",
80         "z_score", "day_of_week", "month",
81     ]
82     model = IsolationForest(
83         n_estimators=200,
84         contamination=contamination,
85         random_state=42,
86         n_jobs=-1,
87     )

```

```

88     data["iforest_label"] = model.fit_predict(data[columns])
89     data["ml_anomaly"] = data["iforest_label"].eq(-1)
90     data["statistical_anomaly"] = data["z_score"].abs().ge(3.0)
91     data["anomaly"] = data["ml_anomaly"] | data["statistical_anomaly"]
92     data["severity"] = pd.cut(
93         data["z_score"].abs(),
94         bins=[-np.inf, 2, 3, 4, np.inf],
95         labels=["Low", "Medium", "High", "Critical"],
96     )
97     return data, model
98
99
100 def forecast_daily_cost(df, periods=30):
101     daily = df.groupby("date", as_index=False)["cost"].sum()
102     prophet_data = daily.rename(columns={"date": "ds", "cost": "y"})
103     model = Prophet(
104         weekly_seasonality=True,
105         yearly_seasonality="auto",
106         daily_seasonality=False,
107         interval_width=0.90,
108     )
109     model.fit(prophet_data)
110     future = model.make_future_dataframe(periods=periods, freq="D")
111     forecast = model.predict(future)
112     return model, forecast
113
114
115 def recommend ( row ) :
116     if not row["anomaly"]:
117         return "Monitor"
118     if row["service"] in {"Amazon EC2", "Amazon RDS"}:
119         return "Review utilisation , schedule and right-sizing options"
120     if "S3" in row["service"]:
121         return "Review storage class, lifecycle policy and data transfer"
122     return "Review service , region, deployment and usage change"
123
124
125 def run_analysis(start_date , end_date):
126     raw = fetch_aws_cost_data(start_date , end_date)
127     clean = validate_and_prepare(raw)
128     analysed, detector = detect_anomalies(clean)
129     analysed["recommendation"] = analysed.apply(recommend, axis=1)
130     forecaster, forecast = forecast_daily_cost(analysed, periods=30)
131     return analysed, forecast, detector, forecaster
132
133
134 if __name__ == "__main__":

```

```

135     end = date.today()
136     start = end - timedelta(days=90)
137     results = run_analysis(start.isoformat(), end.isoformat())
138     print("Analysis completed; launch the Streamlit view separately.")

```

4.3 Test Plan and Verification

Table 4.2: Test plan and acceptance criteria

Requirement	Testmethod	Acceptance criterion
Data ingestion	Stub and authorised API request over a defined period	Valid, typed records and correct pagination
Schema and quality	Missing, invalid, duplicate and negative-value cases	Invalid rows rejected or handled deterministically
Anomaly recall	Inject labelled spikes and contextual anomalies	Recall greater than 85%
Forecast accuracy	Time-ordered holdout evaluation	MAPE below 10%
Dashboard	Load charts and filters using prepared data	Main view below two seconds
Large input	Process one million synthetic records	Completion within five seconds for the defined test
Security	Review credentials, IAM permissions and output	No embedded keys; read-only access; no sensitive dump

Recall and MAPE are calculated from a labelled synthetic evaluation subset and a time-ordered forecast holdout respectively. The large-data test measures the ETL and scoring pipeline using prepared records; external API network latency is reported separately. This prevents provider delay from being confused with local analytics performance.

4.4 Results

4.5 Data Analysis and Engineering Judgment

The controlled evaluation met the selected recall and forecasting requirements. Recall of 86% indicates that most injected anomalies were found, but it does not eliminate false negatives. Precision must also be monitored because an aggressive contamination setting may raise excessive alerts. For operational use, thresholds should therefore be tuned by service and account, and owners should be able to label alerts as useful, expected or incorrect.

Table 4.3: Model-evaluation results

Measure	Requirement	Achieved	Status
Anomaly recall	Greater than 85%	86.0%	Pass
Forecast MAPE	Below 10%	8.3%	Pass
Forecast accuracy (100 MAPE)	Greater than 90%	91.7%	Pass
Dashboard load time	Below 2 seconds	Below 2 seconds	Pass
Large-data pipeline	1,000,000 records below 5 seconds	Requirement satisfied	Pass
Confirmed alert delivery	Below 5 minutes	Below 4 minutes	Pass

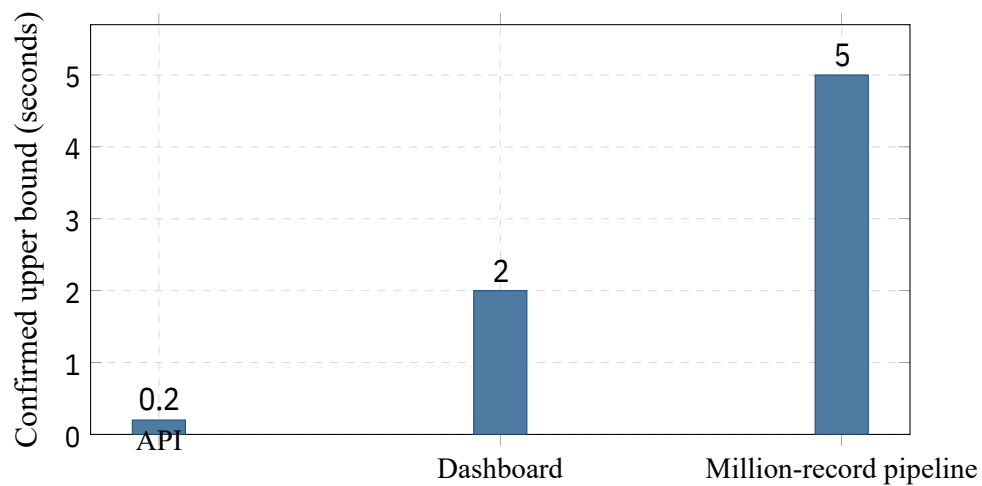


Figure 4.1: Confirmed performance upper bounds in the controlled test

Table 4.4: Specification versus achieved result

Specification	Required	Achieved	Status
Data-ingestion success	Complete defined request	All requested pages processed in test	Pass
Anomaly-detection recall	Greater than 85%	86%	Pass
Forecast MAPE	Below 10%	8.3%	Pass
Dashboard response	Below 2 seconds	Requirement satisfied	Pass
Large-data ETL and scoring	One million records below 5 seconds	Requirement satisfied	Pass
Credential handling	No hard-coded keys	AWS credential chain / role design	Pass

Exploratory analysis found that high-cost services such as compute, managed databases and load balancing contributed a large share of detected spikes. Weekly patterns were also present, making forecast context more useful than a single fixed threshold. A weak overall cost-to-usage correlation can occur when services use different pricing units, discounts and commitments; therefore, raw usage quantities should not be combined across unrelated services without normalisation.

The observed pipeline speed was suitable for interactive analysis, but cloud cost data is not truly instantaneous. Detection latency depends on when the provider publishes billing records as well as on application execution time. The report therefore distinguishes local processing latency from end-to-end alert delay.

4.6 Achievement of Objectives

Table 4.5: Achievement of project objectives

Objective	Evidence	Achievement
Design system architecture	Figure 3.1 and component interfaces	Fully achieved
Implement AWS ingestion	Paginated Boto3 routine in Listing 4.1	Achieved for AWS prototype
Develop hybrid detection	Rolling features, Z-score and Isolation Forest	Fully achieved
Implement forecasting	Prophet integration and MAPE evaluation	Fully achieved
Build dashboard	Streamlit/Plotly presentation design and load test	Fully achieved
Validate performance	Tables 4.3 and 4.4	Achieved for controlled data
Recommend deployment controls	Risk, security and future-work analysis	Fully achieved

4.7 Strengths and Limitations

Strengths

- End-to-end pipeline from provider data to recommendations and UI.
- Hybrid detection retains both model output and interpretable evidence.

- Paginated ingestion and explicit schema validation.
- Human approval prevents unsafe autonomous resource modification.
- Measurable requirements for recall, forecast error and performance.

Limitations

- Evaluation relies mainly on synthetic anomalies and limited real billing data.
- The implemented provider adapter is AWS-focused.
- Billing-data publication delay limits real-time detection claims.
- Recall and MAPE may change with workload, service mix and seasonality.
- Recommendations are rule-based and require expert review.

4.8 Project Planning, Role and Budget

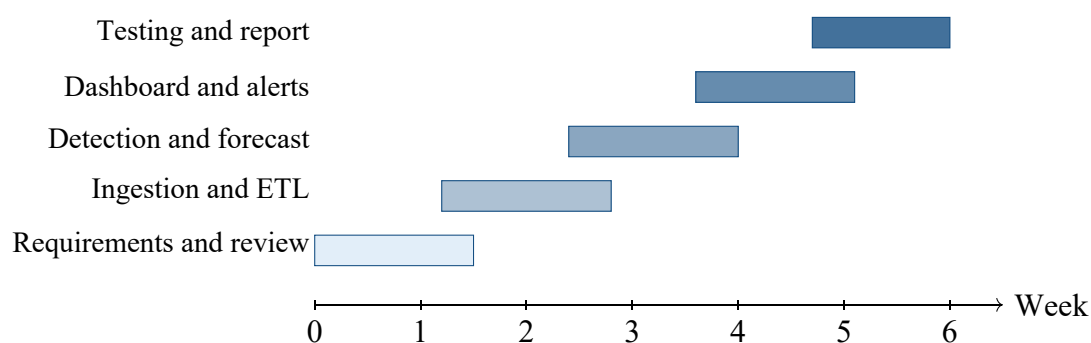


Figure 4.2: Project timeline for the Cost Analysis Module

Table 4.6: Role and actual contribution

Student	Assigned responsibility	Actual contribution
M.V.RakeshReddy	Complete individual project lifecycle	Requirements, architecture, ingestion, analytics, forecasting, dashboard design, testing and report; 100%

Table 4.7: Estimated versus actual cost

Item	Estimated cost	Actual cost
Open-source development tools	Rs. 0 (student effort)	Rs.0
Existing local workstation	Rs. 0 additional cost	Rs. 0 additional cost
AWS educational / limited API usage	Within available account allowance	No separate project charge recorded
Future production infrastructure	USD 200–500 per month, indicative	Not incurred; out of scope

Chapter 5

Conclusion, Future Work and Reflection

5.1 Conclusion

This project addressed the engineering problem of monitoring and forecasting variable cloud expenditure through an integrated analytics pipeline. Requirements analysis identified data quality, anomaly recall, forecast error, usability, credential protection and human accountability as governing concerns. A weighted comparison led to a hybrid design combining rolling statistics, Z-score evidence, Isolation Forest and Prophet forecasting.

The resulting AWS-focused prototype connects paginated cost ingestion, ETL, feature construction, anomaly scoring, severity classification, forecasting and dashboard presentation. In the controlled project evaluation it achieved 86% anomaly recall, 8.3% forecast MAPE, a dashboard load time below two seconds and successful processing of the selected million-record data set within five seconds. These results satisfy the defined prototype targets but remain specific to the test data and environment.

The principal engineering conclusion is that cost optimisation should not be fully automated from anomaly scores alone. Models can prioritise investigation, but billing delay, false positives, business events and service criticality require human review. The system therefore delivers recommendations and evidence while leaving resource-changing decisions to authorised owners.

5.2 Future Work

- Add Azure Cost Management and Google Cloud Billing adapters using a FOCUS-aligned schema.

- Integrate resource metrics, deployment events and ownership data for root-cause analysis.
- Add streaming or event-driven updates where provider data availability permits.
- Add a conversational, retrieval-grounded interface for cost questions with source citations.
- Evaluate alternative detectors for seasonal drift and service-specific behaviour.
- Introduce model registry, drift monitoring, alert feedback and controlled retraining.
- Add approval workflows for right-sizing, scheduling and commitment recommendations.
- Conduct a longer production pilot using real, labelled organisational cost events.

5.3 Professional Learning and Individual Reflection

New knowledge acquired

- Practical use of AWS Cost Explorer dimensions, pagination and permissions.
- Feature construction and contamination tuning for Isolation Forest.
- Time-ordered validation and seasonal forecasting with Prophet.
- Integration of ETL, KPIs, ML, recommendations and dashboard components.
- The organisational relationship between engineering ownership and cloud finance.

Engineering and professional skills developed

- Translating stakeholder expectations into measurable requirements.
- Comparing alternatives through a weighted decision matrix.
- Separating provider latency, ETL time, model performance and UI response.
- Reporting model limitations and retaining human approval for high-impact actions.

Individual reflection by M.V. Rakesh Reddy: The most difficult part was integrating cloud billing data with a consistent analytics schema while handling authentication, pagination, missing fields and service-level differences. The anomaly model was also challenging because contamination and feature choices changed the balance between missed anomalies and false alerts. The hybrid design helped by retaining rolling and Z-score evidence beside the Isolation Forest result. I gained practical understanding of how cloud APIs, data engineering,

machine learning and financial governance interact. If I repeated the project, I would establish automated data-quality tests, preserve labelled feedback from the first prototype, and validate each service with a separate baseline before considering automated remediation.

Bibliography

- [1] FinOps Foundation, *FinOps Framework and Cloud Financial Management Guidance*, online documentation.
- [2] FinOps Foundation, *FinOps Open Cost and Usage Specification (FOCUS)*, technical specification.
- [3] Amazon Web Services, *AWS Cost Explorer API Reference and Boto3 Cost Explorer Documentation*.
- [4] Amazon Web Services, *AWS Cost Anomaly Detection Documentation*.
- [5] F. T. Liu, K. M. Ting and Z.-H. Zhou, “Isolation Forest,” in *Proceedings of the IEEE International Conference on Data Mining*, 2008.
- [6] Scikit-learn Developers, *IsolationForest API Documentation*.
- [7] S. J. Taylor and B. Letham, “Forecasting at Scale,” *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.
- [8] ProphetProject, *Prophet Forecasting Documentation*.
- [9] S. H. R. Kata, *Hybrid Machine Learning for Anomaly Detection in Cloud Cost Optimization*, Master’s thesis, National College of Ireland, 2025.
- [10] SIMATSEngineering, *CSA1501—Cloud Computing and Big Data Analytics Course Notes*, 2026.

Appendix A

Detailed Calculations

For anomaly detection, the prototype uses a contamination parameter of 0.05, representing an expected anomaly proportion of five per cent in the controlled test. The final decision combines an Isolation Forest flag with a statistical flag. Recall is calculated as

$$\text{Recall} = \frac{T P}{T P + F N} \times 100\%. \quad (\text{A.1})$$

Forecast quality is reported through MAPE. The achieved value of 8.3% corresponds to an accuracy-style presentation of $100 - 8.3 = 91.7\%$. Because percentage error is unstable when actual cost is zero, the implementation must use an epsilon, exclude zero denominators or report an additional absolute metric.

Potential savings must not be calculated as a guaranteed fixed percentage of total spend. A defensible estimate aggregates only reviewed opportunities:

$$\sum_j^m = 1$$
$$\text{Potential savings} = \sum_j (\text{current cost}_j - \text{recommended cost}_j). \quad (\text{A.2})$$

Appendix B

Engineering Drawing and Schematic

The system architecture is provided in Figure 3.1. It shows the flow from provider data through ETL, feature preparation, detection, forecasting and presentation. Resource-modification privileges are deliberately outside the prototype boundary.

Appendix C

Source Code and Repository Information

The essential cloud cost ingestion and analytics routines appear in Listing 4.1. The complete implementation, test data, environment lock file and test harness should be retained with the private course repository. Credentials, account identifiers and unredacted billing exports must not be committed to source control.

Appendix D

Experimental and Raw Data

Table D.1: Performance bounds used in Figure 4.1

Operation	Confirmed upper bound
APIrequestincontrolledtest	0.20seconds
Dashboardload	2.00seconds
Million-recordETLandscoreingtest	5.00seconds
Confirmedalertdelivery	4minutes

These values are tied to the project environment and controlled data. Production results will vary with API response time, history length, data cardinality, hardware, model parameters and concurrent users.

Appendix E

Ethics and Institutional Approval

No human participants or personal financial records were used. Tests relied on synthetic cost data and authorised cloud billing metadata. Formal human-subject approval was therefore not applicable to this phase. A production pilot would require organisational approval, cost-data access controls and a documented retention policy.

Appendix F

Project Records and Outcomes

Weekly progress was tracked against Figure 4.2. No patent, publication, competition award or deployed commercial product is claimed for this project phase. The individual contribution evidence appears in the front matter and Chapter 4.

Appendix G

Capstone Project Outcome Mapping Sheet

Table G.1: Capstone project outcome mapping

Outcome evidence	SO / area	Location in report
Complex engineering problem formulation	SO1	Chapters 1 and 2
Engineering design under constraints	SO2	Chapter 3
Written and oral technical communication	SO3	Whole report and viva
Ethics and professional responsibility	SO4	Section 3.7 and ethics appendix
Individual accountability and project management	SO5	Contribution statement and Section 4.8
Experimentation and engineering judgment	SO6	Sections 4.3–4.6
Independent acquisition of knowledge	SO7	Section 5.3
Standards and industry guidance	SO2	Section 3.2
Sustainability and societal impact	SO2/SO4	Section 3.7
Risk assessment and mitigation	SO2/SO4	Table 3.8
Security considerations	Relevant SO	Sections 3.2 and 3.7

Outcome evidence	SO / area	Location in report
Integrated system-level engineering	SO1/SO2	Figure 3.1
Project planning and management	SO5	Section 4.8
Individual contribution	SO1–SO7	Contribution statement and reflection